

Lab 3 - Week 5. Mutation Testing

Aim

In this lab, we write our own mutation testing framework like FitSpec. We manipulate the output data of a function under test, to check that any incorrect output for a given input fails the tests. This way, we can discover survivors, the minimal set of properties, and find all the conjectures.

For this lab, we use a 'multiplication table' implementation+properties as target for mutation testing. The related Haskell code can be found in *MultiplicationTable.hs*.

NOTES:

1. Testing with QuickCheck is not mandatory for this lab.
 2. The provided `mutate'` function provides a good foundation which can be integrated in your work.
 3. The `mutate` function is very similar to `mutate'` with the exception of its types, as it utilises the monadic Maybe types. While this distinction can serve as a source of inspiration, it is not mandatory for implementation.
-

Prerequisites

Submission guidelines

Files

- Submit one Haskell file per exercise.
- Name files ExerciseX.hs for exercises and EulerX.hs for Euler problems.
- Each module must be named to match: module ExerciseX where.
- Follow naming conventions exactly — some exercises are auto-graded.
- Do not include personally identifiable information.

- Indicate additional dependencies in a comment at the top of the file.
- Record time spent:
 - Time Spent: X min (in minutes).
- Codegrade: create a new group with all teammates for each assignment.
- Include a txt file containing the report on design choices
- You are not allowed to use GenAI to solve the proposed exercises, but you can use it as a support tool. If you use GenAI, write a comment including how you used it.
- Take a look at the paper [FitSpec: refining property sets for functional testing](#).
- Look at the example code in the `Mutation.hs` file.
- Take a look back at [the weaker/stronger theory of the Hoare lab](#).
- Look at the example code in the `FitSpec.hs` file and run the *main* function.
- Include a txt file containing the report on design choices
- You are not allowed to use GenAI to immediately obtain a solution to the proposed exercises. Any other use of GenAI is subject to the UvA AI policy (<https://www.uva.nl/en/about-the-uva/policy-and-regulations/education/policy-framework-and-guidelines-on-genai-in-education.html>).

Imports

```
import Data.List
import Test.QuickCheck
import Mutation
```

Exercise 1

We provide some mutators to mutate the output of the list in `Mutation.hs`. Write down which types of output are not yet covered by these mutators, and about

their weakness/strength. Come up with a list of other mutators and implement (a subset of) them.

Hint: Why wouldn't it be a good idea for the mutant to be the same as the original output list?

Deliverables: List of mutators and rationale, implementation, oral and written design report, indication of time spent.

Exercise 2

Write a function `countSurvivors` that counts the number of survivors:

```
countSurvivors :: Integer -> ([[Integer] -> Integer -> Property]) -> (Integer -> [Integer]) -> IO Integer
```

Where the first argument is the number of mutants (4000 in the `FitSpec` example), the second argument is the list of properties, and the third argument is the function under test (the multiplication table function in this case).

The output is the number of surviving mutants (0 in the `FitSpec` example).

Document the effect of which mutations are used and which properties are used on the number of survivors.

Hints:

1. Consider the relation between properties and survivors.
2. The above-mentioned function definition is not final. Feel free to modify it, for example by adding the mutations that should be used.

Deliverables: implementation, documentation of approach, effect of using different mutators/properties, oral and written design report, indication of time spent.

Exercise 3

Find the definition of the minimal property subset in the lecture. Implement a function that calculates the minimal property subsets, given a 'function under test' and a set of properties.

Deliverables: implementation, documentation of approach, oral and written design report, indication of time spent.

Exercise 4

Implement a function that calculates the strength of a given set of properties, which is the percentage of mutants they kill.

Deliverables: implementation, documentation of approach, oral and written design report, indication of time spent.

Exercise 5

Implement function(s) that calculate the conjectures: properties that are equivalent, whose cases are subsets of other properties, etc.

Deliverables: implementations, documentation of approach, oral and written design report, indication of time spent.

Exercise 6 (Bonus)

Create a function that we pass the function under test, a set of properties, and a number indicating the number of mutants, that visualizes the results of the functions from the previous exercises.

Deliverables: Implementation, oral and written design report, indication of time spent.
